

LLM GATEWAY

Rate Limiting, Multi-Provider Fallback, and Circuit Breakers for LLM Traffic

Project 12 of the AI Engineering Portfolio | Build Guide & Interview Companion

Author: Anas Amiar

"Every company with more than one team using LLMs ends up building this: one choke point enforcing who may call, how often, at what cost — and what happens when a provider goes down."

How to Use This Guide

This guide documents the project exactly as it was built: a short pitch, the tech stack, a phase-by-phase build order with reasoning for why each phase comes before the next, and the talking points worth memorizing before an interview. Read top to bottom to learn the project from scratch, or jump to the "Interview Talking Point" box at the end for a quick pre-call refresher.

What You're Building

A production-style API gateway in front of all LLM calls: per-team API keys with allowed model tiers, token-bucket rate limits (clean 429s with Retry-After), daily budget caps (warn at 80%, block at 100%), retry-then-fallback routing along per-tier provider chains, and circuit breakers that skip a failing provider instantly instead of burning a timeout per request.

The demo plays a full incident lifecycle in one run: primary provider goes down → 2 retries → fallback serves traffic → circuit opens after 3 failures in 30s → provider recovers → half-open probe → circuit closes → traffic returns to the primary. **Zero failed user requests during the outage.**

WHY THIS PROJECT LANDS INTERVIEWS

This is pure infrastructure engineering applied to AI — exactly the skill set mid-level SWEs already have and AI teams desperately need. Rate limiting, budget enforcement, failover, and circuit breaking are the operational problems every scaled AI org hits; leading with reliability numbers positions you as someone who ships infrastructure, not demos.

Tech Stack

Layer	Tool / Library	Purpose
Language	Python 3.11+	Core implementation
Data validation	Pydantic v2	TeamConfig, GatewayRequest/Response, ProviderHealth
Rate limiting	Token bucket per team	Same algorithm as Redis-backed production
Budgets	Per-team daily caps	Computed from actual per-call costs; warn 80% / block 100%
Fallback routing	Per-TIER provider chains	Policy survives model lineup changes
Circuit breakers	closed/open/half-open	3 failures in 30s opens; 60s cooldown; probe closes
Providers (mock)	Controllable failure modes	forced_down switch makes outages demoable
Clock	Caller-supplied 'now'	4-minute incident timeline replayed in milliseconds

The Core Flow

handle(request, now) → auth (API key → team config) → tier check → token-bucket rate limit → budget check → routed call: for each provider in the tier's chain, skip if circuit open, retry x2 on primary only, one shot per fallback → record cost + metrics.

Step-by-Step Build Guide

Five components, resilience-first: providers with controllable failure modes came first because features you can't trigger on demand can't be tested or demonstrated.

PHASE 1: Providers with Controllable Failure Modes

Day 1-2

Goal: Make outages demoable — resilience you can't trigger is resilience you can't test.

- Mock providers (openai/anthropic/ollama-local) with latency, cost, a baseline 2% transient error rate, and a forced_down switch.
- Fallback chains per TIER (standard/premium/local-only), not per model — routing policy survives lineup changes.

Deliverable: PROVIDERS + TIER_CHAINS with on-demand outage simulation.

PHASE 2: Token-Bucket Rate Limiting

Day 2-4

Goal: Per-team isolation: one team's spike can't starve another.

- One bucket per team, refilled continuously by elapsed time against a caller-supplied clock.
- Empty bucket → clean 429 with a computed Retry-After header value.
- Verified: 8 rapid requests against a 5/min limit → exactly 5 pass, 3 rejected.

Deliverable: RateLimiter.allow(team, rate, now) — Redis-ready algorithm.

PHASE 3: Budget Enforcement

Day 4-6

Goal: Finance-grade cost control per team.

- Spend tracked from actual per-call costs at serve time.
- Warning emitted at 80% of the daily budget; hard block at 100% with a clear, actionable error message.

Deliverable: Per-team budget caps wired into the request path.

PHASE 4: Circuit Breakers + Fallback Routing

Day 6-9

Goal: Fail over fast, stop paying timeouts to dead providers, recover automatically.

- Three-state breaker per provider: closed → open (3 failures/30s) → half-open (60s cooldown) → closed on probe success. Every transition logged.
- Open circuit = provider skipped instantly, no timeout burned.
- Retries (2, with backoff) only on the PRIMARY — fallbacks get one shot each; retrying everything would amplify latency for doomed requests.

Deliverable: CircuitBreaker + the routed-call loop in the gateway core.

PHASE 5: The Incident Lifecycle Demo

Day 9-12

Goal: The whole story in one deterministic run.

- Scene 1: normal traffic on the primary.
- Scene 2: rate limiting — 5/8 pass, 3 clean 429s.
- Scene 3: outage — fallback serves 100% of traffic, circuit opens.
- Scene 4: recovery — half-open probe closes the circuit, printed state log.
- Scene 5: budget exhaustion — blocked with the exact spend in the error.

Deliverable: `python3 -m gateway.demo` — zero failed user requests through a full outage.

Demo Results

Scenario	Result
Provider outage	100% of requests served via fallback — zero user-facing failures
Circuit breaker	opened after exactly 3 failures; auto-recovered via half-open probe
Rate limiting	5/8 rapid requests passed a 5/min limit; 3 got 429 + Retry-After
Budget cap	team blocked at \$0.021 spend on a \$0.02 budget, warning fired at 80%
Breaker log	closed->open (t=8.8s), open->half_open (t=81.8s), half_open->closed

Architecture Decisions

Why circuit breakers when you already have fallbacks?

Fallbacks answer 'where else can this request go?' Breakers answer 'why are we still paying a timeout to ask a dead provider?' Without one, every request during an outage burns a full timeout before failing over.

Why fallback chains per tier instead of per model?

'GPT-4o is down, use Claude' hardcodes today's lineup into routing. Per-tier chains survive model renames and lineup changes.

Why retries only on the primary?

Retrying every provider in the chain multiplies worst-case latency for a request that's probably doomed. Retry where a transient blip is plausible; fail fast everywhere else.

Deliberately Out of Scope for v1

- Real provider SDKs behind the abstraction (interface is in place).
- Redis-backed distributed rate limiting — same algorithm, different store.
- Streaming passthrough with simultaneous logging.
- OpenTelemetry + Prometheus/Grafana export (metrics already collected).
- Priority scheduling (realtime pre-empts batch) and the admin API.

INTERVIEW TALKING POINT

"What was the hardest design decision?"

Where to put retries. The obvious design retries every provider in the chain — but that multiplies worst-case latency for a doomed request and turns the gateway into a latency amplifier during a real outage. The right answer: retry only the primary (where a transient blip is plausible), one shot per fallback, and let the circuit breaker eliminate even the first attempt once a provider is known-dead. The demo shows the effect: the first request of the outage pays 2 retries, and after the circuit opens, subsequent requests hit the fallback with zero retries and zero wasted timeouts.

What I'd Build Next

- Redis-backed rate limiting for multi-instance deployment.
- Streaming passthrough with buffer-for-logging.
- OpenTelemetry spans + the three Grafana dashboards (ops/business/performance).
- Priority scheduling under contention; the no-restart admin API.

- Integration with the semantic cache (Project 8): cache first, gateway on miss.

Companion Projects in This Portfolio

The operational shell around the rest of the portfolio: the LLM Cost Autopilot (Project 2) decides which model a request deserves, the Semantic Cache (Project 8) eliminates repeat calls, and this gateway enforces who may call, how often, at what cost, and what happens when providers fail. Together: the LLM serving stack.

Repository: github.com/Anas-Amiar/Project-12-llm-gateway